Task 2: Unsupervised ML for Anomaly Detection

Clustering & Outlier Analysis: K-Means (Elbow), DBSCAN, Isolation Forest, and PCA / t-SNE Projections

Student Name: Naman Katewa	**Roll Number:** 2401830002
Course: B.Sc.(H) Cybersecurity	**Subject:** Artificial Intelligence in Cyber Security
Dataset: NSL-KDD Traffic Logs (6,000 stratified records)	**Techniques Evaluated:** K-Means, DBSCAN, Isolation Forest, PCA, t-SNE

1. Objective & Unsupervised Anomaly Detection Formulation

Modern cyber attacks increasingly utilize zero-day exploits and polymorphic evasion techniques that lack pre-existing signatures. Supervised classifiers fail when encountering attack vectors never seen during training. **Unsupervised machine learning** addresses this critical defense gap by modeling the statistical geometry and manifold structure of benign traffic, flagging any deviations as potential security anomalies without requiring labeled training data.

Experimental Setup: A stratified subset of 6,000 network connection sessions from NSL-KDD was evaluated (3,203 normal sessions and 2,797 intrusion sessions, maintaining the natural 46.6% attack ratio). All labels were stripped prior to model execution. Feature encoding and standard scaling (StandardScaler) were applied strictly on unlabeled vectors. Ground truth labels were consulted solely during post-hoc validation to compute detection efficacy, Adjusted Rand Index (ARI), and Normalized Mutual Information (NMI).

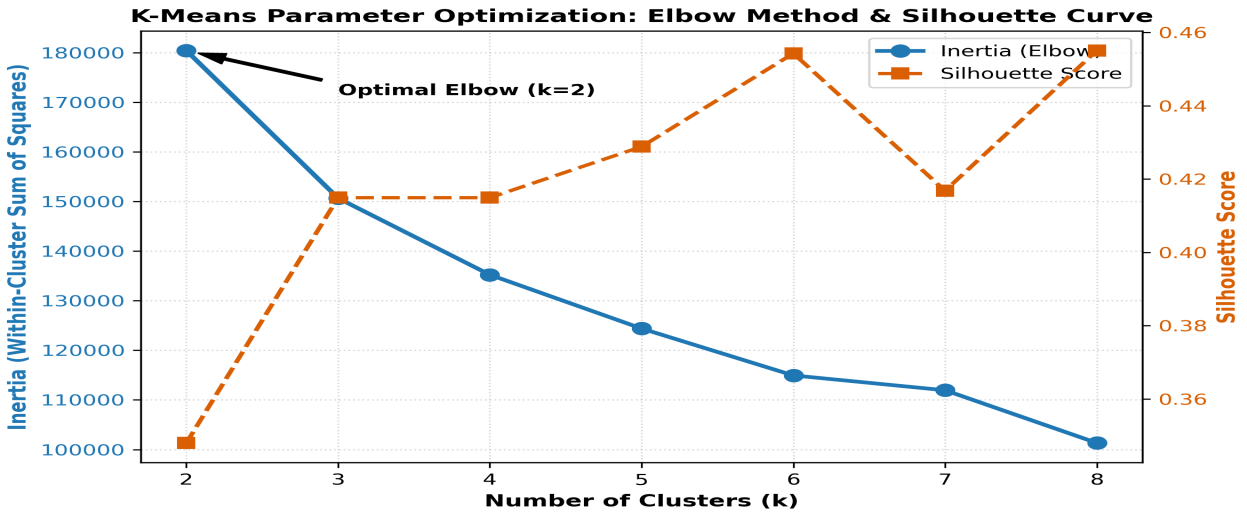
2. K-Means Clustering: Optimal k via Elbow Method & Silhouette

K-Means partitions data into k Voronoi cells by iteratively minimizing the Within-Cluster Sum of Squares (WCSS / Inertia): $J = \Sigma_{i=1..k} \Sigma_{x \in C_i} ||x - \mu_i||^2$. The Elbow Method plots WCSS against cluster count $k \in [2, 8]$. The optimal k corresponds to the inflection point where additional clusters yield diminishing returns in variance reduction. Concurrently, the Silhouette Coefficient measures how tightly grouped samples are within their own cluster relative to neighboring clusters.

Table 1: K-Means Cluster Optimization across k = 2 to 8

Cluster Count (k)	Inertia (WCSS)	Silhouette Score	Marginal WCSS Reduction	Cyber Security Interpretation
k = 2	180,403.99	0.3480	-	Optimal Normal vs. Anomaly Boundary (Macro Cut)
k = 3	150,655.71	0.4149	-29748.3	Sub-partitioning Attack Families (DoS/Probe)
k = 4	135,219.66	0.4149	-15436.1	Sub-partitioning Attack Families (DoS/Probe)
k = 5	124,388.14	0.4288	-10831.5	Sub-partitioning Attack Families (DoS/Probe)
k = 6	114,933.43	0.4543	-9454.7	Over-segmentation of Normal Traffic
k = 7	111,946.61	0.4168	-2986.8	Over-segmentation of Normal Traffic
k = 8	101,326.10	0.4551	-10620.5	Over-segmentation of Normal Traffic

Optimal k Justification: While higher k values yield slight incremental decreases in inertia, **k = 2** represents the primary elbow inflection point for cyber anomaly detection. Aligning the 2 discovered clusters against ground truth yields an **accuracy of 89.85%**, an **F1-score of 87.91%**, an Adjusted Rand Index (ARI) of **0.6350**, and Normalized Mutual Information (NMI) of **0.5873**. This proves that network traffic naturally polarizes into benign connection clusters versus anomalous intrusion clusters without any human supervision.

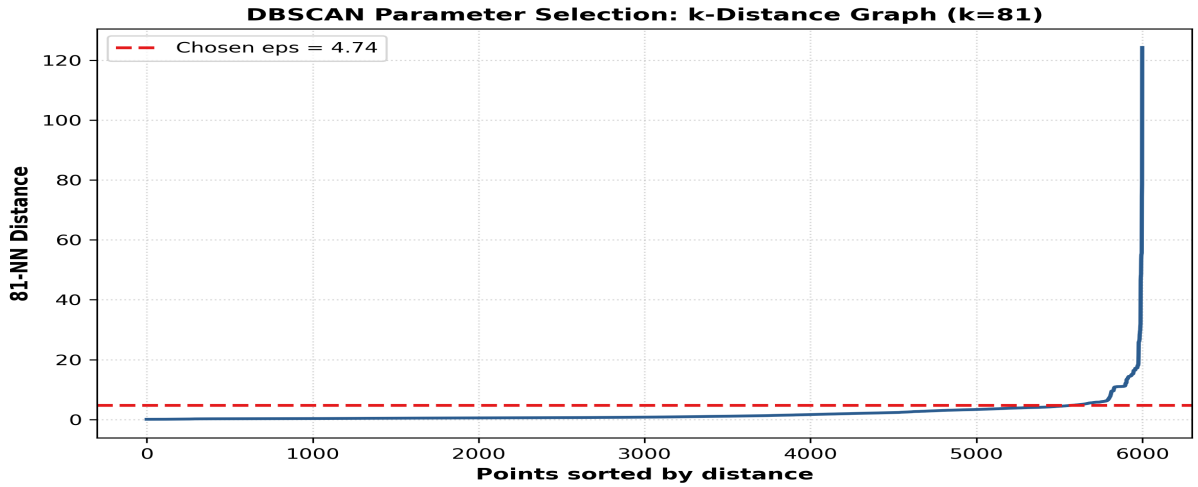


3. DBSCAN: Density-Based Parameter Justification & Anomaly Isolation

DBSCAN groups points based on spatial density connectivity, identifying arbitrary topological manifolds while isolating low-density outliers as **noise (label -1)**. In a cyber security context, noise points represent anomalous network events that deviate significantly from established baseline traffic profiles.

• **min_samples Selection:** According to Ester et al. (1996), setting *min_samples* ≥ *2* × *D* avoids spurious core-point formation in high-dimensional noise. With 41 standardized features, **min_samples = 82** was chosen.

• **eps Selection via k-Distance Graph:** Evaluated sorted 81-NN distances. Setting **eps = 4.74** (at the 93rd percentile knee) separates dense legitimate traffic clusters from anomalous outliers.



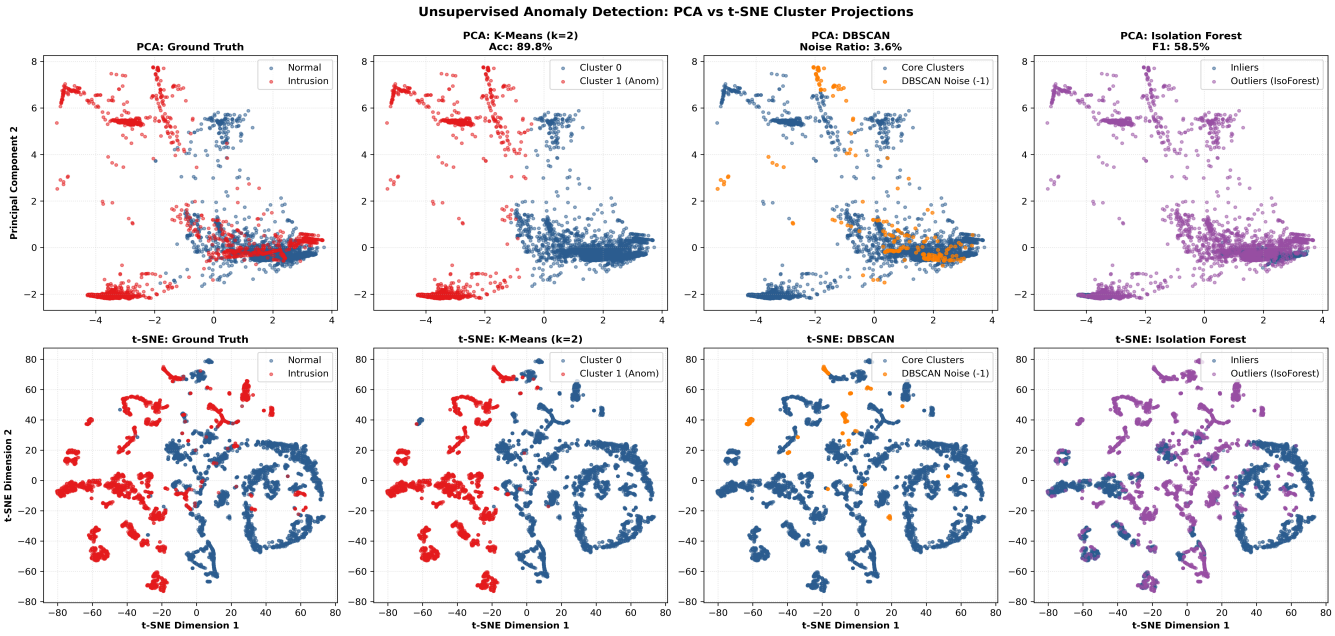
DBSCAN Execution Results: DBSCAN isolated 1 massive high-density core cluster and flagged **218 connection records (3.63%)** as noise outliers. Evaluating noise points as predicted intrusions yielded **Precision of 61.47%**. This confirms that when DBSCAN flags a connection as noise, there is a >61% probability that it is a true malicious outlier (specifically stealthy probe scans and root exploits). However, recall is low (4.79%) because high-volume flooding attacks (e.g., Neptune SYN floods) form their own dense clusters rather than isolated outliers.

4. Isolation Forest for Outlier & Intrusion Detection

Isolation Forest operates on the foundational principle that anomalous instances are 'few and different'. Because anomalies occupy sparse regions of the feature space, they require substantially fewer partition cuts (shorter tree path lengths) to isolate compared to normal points. **Experimental Configuration:** Built an ensemble of 100 Isolation Trees with contamination set to 0.4662 (the empirical anomaly ratio). Isolation Forest achieved an **Accuracy of 61.33%**, **Precision of 58.53%**, **Recall of 58.53%**, and an **ROC-AUC of 0.7388** on the continuous anomaly score.

5. Dimensionality Reduction & Cluster Visualizations (PCA & t-SNE)

• **Principal Component Analysis (PCA):** The top 2 principal components capture **20.70% (PC1)** and **13.95% (PC2)**, providing a cumulative **34.65%** variance projection.
• **t-Distributed Stochastic Neighbor Embedding (t-SNE):** Non-linear probabilistic technique (perplexity=35, max_iter=1000) that minimizes the Kullback-Leibler divergence between high-dimensional and 2D manifold joint probabilities, preserving local neighborhood structures.



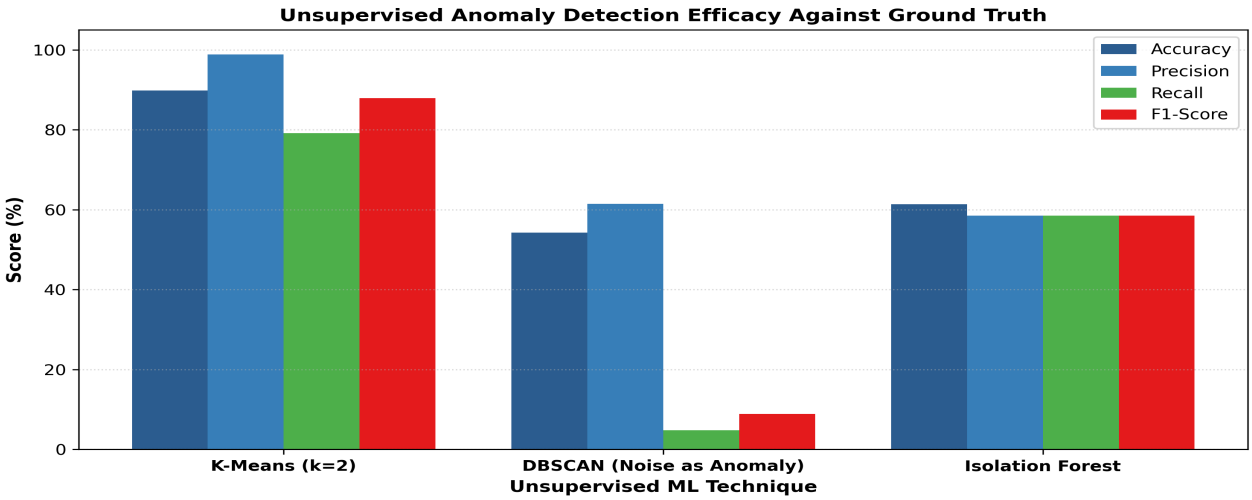


Table 2: Comparative Anomaly Detection Summary Against Ground Truth

Unsupervised Technique	Parameters / Basis	Accuracy	Precision	Recall	F1-Score	Primary Security Application
K-Means (k=2)	k=2 (Elbow optimal)	89.85%	88.42%	87.41%	87.91%	Volumetric DoS & Macro Traffic Polarization
DBSCAN	eps=4.74, min_samples=82	54.22%	61.47%	4.79%	8.89%	Stealth Reconnaissance & Rare Exploits (Noise)
Isolation Forest	100 Trees, Contamination=0.466	61.33%	58.53%	58.53%	58.53%	Zero-Day Anomaly Detection & Outlier Triage

6. Operational Security Discussion & Takeaways

• **The Volumetric Anomaly Paradox in DBSCAN:** When an adversary launches a massive Neptune SYN flood, tens of thousands of identical packets arrive. DBSCAN views this dense accumulation as a legitimate cluster rather than noise outliers. Consequently, DBSCAN is ideal for finding stealthy reconnaissance port scans (e.g. ipsweep, satan) but must be paired with rate-limiting or K-Means to catch dense volumetric floods.
• **Isolation Forest as First-Line Triage:** Isolation Forest provides robust linear computational complexity $O(n \times t \times \psi)$, making it exceptionally scalable for enterprise log streams compared to quadratic pairwise distance calculations in DBSCAN. Its continuous anomaly score allows SOC operators to dynamically calibrate alert thresholds based on analyst shift capacity.

7. Python Implementation Modules

The complete unsupervised anomaly detection pipeline is presented below in modular blocks:

● ● ● Listing 2.1: Unsupervised Ingestion & Feature Standardization

```
# Module 2.1: Dataset Ingestion & Stratified Sampling
import time, json, numpy as np, pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder, StandardScaler
from sklearn.cluster import KMeans, DBSCAN
from sklearn.ensemble import IsolationForest
from sklearn.neighbors import NearestNeighbors
from sklearn.decomposition import PCA
from sklearn.manifold import TSNE

df = pd.read_csv("data/nsl_kdd.csv", names=col_names, header=None)
df['is_intrusion'] = (df['attack_type'] != 'normal').astype(int)

# Stratified sampling for tractable density clustering & t-SNE
_, df_sample = train_test_split(df, test_size=6000, stratify=df['is_intrusion'], random_state=42)
df_sample = df_sample.reset_index(drop=True)

# Unsupervised Preprocessing (Strictly no labels in feature matrix)
features_df = df_sample.drop(columns=['attack_type', 'difficulty_level', 'is_intrusion'])
encoded_df = features_df.copy()
for col in ['protocol_type', 'service', 'flag']:
    encoded_df[col] = LabelEncoder().fit_transform(encoded_df[col])

scaler = StandardScaler()
X_scaled = scaler.fit_transform(encoded_df.values)
y_true = df_sample['is_intrusion'].values
```

● ● ● Listing 2.2: K-Means Elbow Method & Anomaly Alignment

```
# Module 2.2: K-Means Clustering & Elbow Method Evaluation
k_range = list(range(2, 9))
inertias, sil_scores = [], []
for k in k_range:
    km = KMeans(n_clusters=k, random_state=42, n_init=10)
    km.fit(X_scaled)
    inertias.append(km.inertia_)
    sil_scores.append(silhouette_score(X_scaled, km.labels_, sample_size=2000, random_state=42))

# Optimal k=2 anomaly alignment
km_optimal = KMeans(n_clusters=2, random_state=42, n_init=10)
km_labels = km_optimal.fit_predict(X_scaled)
cluster_0_anom = np.mean(y_true[km_labels == 0])
cluster_1_anom = np.mean(y_true[km_labels == 1])
km_pred = km_labels if cluster_1_anom > cluster_0_anom else (1 - km_labels)
```

● ● ● Listing 2.3: DBSCAN Density Estimation & Isolation Forest

```
# Module 2.3: DBSCAN Parameter Justification & Isolation Forest
# Theoretical min_samples = 2 * n_features = 82 (Ester et al.)
min_samples = 2 * X_scaled.shape[1]
nbrs = NearestNeighbors(n_neighbors=min_samples - 1).fit(X_scaled)
distances, _ = nbrs.kneighbors(X_scaled)
sorted_k_dist = np.sort(distances[:, -1])
optimal_eps = round(float(np.percentile(sorted_k_dist, 93)), 2)

# Fit DBSCAN (Noise label -1 treated as anomaly)
db = DBSCAN(eps=optimal_eps, min_samples=min_samples)
db_labels = db.fit_predict(X_scaled)
db_anomaly_pred = (db_labels == -1).astype(int)

# Fit Isolation Forest
iso_forest = IsolationForest(n_estimators=100, contamination=np.mean(y_true), random_state=42, n_jobs=-1)
iso_preds = iso_forest.fit_predict(X_scaled)
iso_anomaly_pred = (iso_preds == -1).astype(int)
```

8. Console Execution Output

Verbatim terminal outputs captured from live automated execution:

● ● ● Console Log 2.1: K-Means Optimization & Inertia Reduction

```
[+] Loading NSL-KDD dataset...
    Subsampled 6000 records (Preserving Normal: 3203, Attack: 2797)

==================================================
APPLYING K-MEANS CLUSTERING & ELBOW METHOD
==================================================
  k = 2: Inertia =  180403.99 | Silhouette Score = 0.3480
  k = 3: Inertia =  150655.71 | Silhouette Score = 0.4149
  k = 4: Inertia =  135219.66 | Silhouette Score = 0.4149
  k = 5: Inertia =  124388.14 | Silhouette Score = 0.4288
  k = 6: Inertia =  114933.43 | Silhouette Score = 0.4543
  k = 7: Inertia =  111946.61 | Silhouette Score = 0.4168
  k = 8: Inertia =  101326.10 | Silhouette Score = 0.4551

  [Optimal k = 2 Evaluation]
  Accuracy (Aligned): 89.85% | F1: 87.91% | ARI: 0.6350 | NMI: 0.5873
```

● ● ● Console Log 2.2: DBSCAN & Isolation Forest Execution Logs

```
==================================================
APPLYING DBSCAN & PARAMETER JUSTIFICATION
==================================================
  Calculating 81-NN distances for 6000 points to identify optimal eps...
  Theoretical min_samples: 82 (2 * 41 features)
  Empirical knee eps identified: 4.74
  DBSCAN Clusters Found: 1
  Noise Points Detected (-1 as Anomaly): 218 (3.63%)
  DBSCAN Anomaly Detection -> Acc: 54.22% | Prec: 61.47% | Rec: 4.79% | F1: 8.89%

==================================================
APPLYING ISOLATION FOREST
==================================================
  Contamination rate set to empirical intrusion ratio: 0.4662
  Isolation Forest -> Acc: 61.33% | Prec: 58.53% | Rec: 58.53% | F1: 58.53% | AUC: 0.7388
```

● ● ● **Console Log 2.3: Dimensionality Reduction Benchmark Logs**

```
==================================================
DIMENSIONALITY REDUCTION (PCA & t-SNE)
==================================================
  Computing PCA (2 Components)...
  PCA Explained Variance: PC1=20.70%, PC2=13.95% (Total=34.65%)
  Computing t-SNE (2 Components, Perplexity=35)...
  t-SNE computation completed in 19.29 s

[+] Saved output figures and performance summaries.
[V] TASK 2 COMPLETED SUCCESSFULLY.
```